BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION SOFTWARE ENGINEERING

DISSERTATION THESIS

Testing database applications

Supervisor
Assoc. Prof. Habil. Vescan Andreea, PhD

Author
Șoim Adrian

2026

UNIVERSITATEA BABEŞ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
SPECIALIZAREA INGINERIE SOFTWARE

LUCRARE DE DISERTAŢIE

TESTAREA APLICATIILOR CU BAZE DE DATE

Conducător ştiinţific
Conf. Dr. Habil. Vescan Andreea

Absolvent
Șoim Adrian

2026

ABSTRACT

Context: Database management systems (DBMS) represent the critical infrastructure of modern software, making their logical correctness non-negotiable. While property-based fuzzing tools like SQLancer excel at discovering semantic bugs, their native command-line execution limits observability. The lack of history persistence, real-time monitoring, and visual analytics makes large-scale comparative testing an inefficient and error-prone manual task.

Objectives: This thesis bridges the gap between raw test generation and analytical observability. The main objective is the design, development, and evaluation of a custom orchestration platform, capable of transforming isolated database fuzzing into an automated, visually monitored, and highly persistent continuous testing workflow.

Methods: To achieve this, a decoupled three-tier architecture was implemented: an interactive single-page application for real-time visualization, an asynchronous backend for orchestrating OS-level Java subprocesses and capturing I/O streams, and a relational persistence layer for logs and metrics. To validate the platform's efficacy, a replication and extension study was conducted across multiple modern database engines using short-burst fuzzing and various testing oracles.

Results: The platform sustained high-throughput testing environments (peaking at over 16,600 queries per second) and successfully identified 19 unique logical bugs. Notably, 13 structural vulnerabilities were discovered in the TiDB distributed engine using the TLP HAVING oracle. While prior work (Rigger and Su, 2020) [RS20] evaluated TiDB, it did not implement TLP HAVING for this specific engine. Consequently, this thesis evaluates it on a newer TiDB build (v7.5.1) and reports these novel vulnerabilities. Furthermore, comparative analysis showed that highly specific oracles (e.g., TLP) are significantly more effective at triggering deep semantic flaws than high-throughput, non-optimizing alternatives like NoREC.

Conclusion: The major contribution of this thesis is the creation of a scalable observability platform that abstracts the complexities of database fuzzing. By coupling a sophisticated CLI testing tool with a custom real-time orchestration layer, it provides a reusable framework for the engineering community. Enhancing the observability of testing processes simplifies research workflows and directly accelerates the discovery, documentation, and comparative analysis of critical database vulnerabilities.

Contents

Chapter 1

Introduction

Database management systems [SKS20] represent the backbone of modern digital infrastructure, being responsible for storage, manipulation, and data recovery in almost all software systems, from critical banking applications all the way to e-commerce platforms. Ensuring the correctness of these systems is a non-negotiable requirement because a computing error or a wrong data return can have severe consequences. The following work approaches the automated testing problem of databases using different fuzzing techniques and proposes an integrated software solution for orchestrating and monitoring the process. During this research, a web full-stack platform was designed and developed that interfaces with the SQLancer tool, facilitating the discovery of vulnerabilities and the comparative analysis of database engines. The original results include the identification of 19 unique bugs in mature systems and showcase the effectiveness of some testing oracles over others.

1.1 Relevance of the research topic

The relevance of the research topic derives from the increasing complexity of current database management systems. SQL language [RG03] is of declarative nature; the developer specifies what data they want to obtain, but the decision about the extraction (order of execution, index usage, join strategies) methodology is delegated to the query optimizer. These optimizers have evolved into complex software components. Because of the high possible states of a database and the number of permutations of SQL clauses, manual testing or through other predefined unit testing suites has become insufficient.

In this context, logical bugs represent one of the most insidious threats. Unlike a critical system crash, a logic bug does not produces fail-stop errors; the query is executed apparently correct, but the returned result set is incomplete or computed incorrectly. Discovering these defects necessitates advanced fuzzing tools based on

properties, capable of generating massive data volumes and validating the results using logical oracles, a research domain that is in expansion and very important for the software industry [ZCH+20].

1.2 Motivation

Although automated testing tools like SQLancer [RS20] have revolutionized bug detection in database management systems, their usage in research environments or practical development often encounters significant operability barriers. Running the tool is done exclusively through the command line, generating many logs that are difficult to parse and interpret manually. The lack of a persistent environment for storing the execution history makes replication studies and comparative analysis between different engines' performance prone to human error.

The main objective of this work is to eliminate this technological barrier. The need of running these tools in a modern, visual, monitored, and scalable workflow was identified, offering the researchers a dedicated observability platform.

1.3 Objectives

In order to answer the identified needs, this work aims to reach the targets listed below:

1. **Developing an asynchronous orchestration architecture:** Designing a backend server that is capable of launching, monitoring, and stopping the operating system's processes related to the fuzzing tool, efficiently managing the hardware resources.

2. **Creating a centralized observability system:** implementing a relational persistence layer and a dashboard for visualizing the experiments' status, capturing terminal logs and aggregating the performance metrics.

3. **Experimental validation of the platform:** Leading an empirical study on testing a large selection of database systems (SQLite, MySQL, TiDB, MariaDB, DuckDB) to confirm the platforms' capacity to catch real errors.

4. **Comparative evaluation of the testing oracles:** Analyzing the report between the throughput and the efficiency in detecting bugs for the main oracles (TLP, NoREC, PQS).

1.4 Contributions

The original contribution of this thesis is divided into two categories: software engineering and empirical research. At the technological level, the major contribution is represented by designing and implementing from scratch the full-stack orchestration platform (using technologies like React, Python, and SQLite). This platform abstracts SQLancer's complexity and provides a reusable tool for the academic community. On the empirical research level, the thesis contributes through the discovery of 19 unique bugs in the included database engines. A novel contribution of this thesis is the identification of the distributed TiDB system when processing the post-aggregation predicates; applying the TLP HAVING oracle (executed here on TiDB v7.5.1) led to the identification of 13 unique structural bugs. Rigger and Su [RS20] evaluated TiDB in their broader study (as of May 2020) but did not run the HAVING oracle for TiDB in their reported experiments; this thesis therefore provides the first reported results for TLP HAVING on a more recent TiDB build.

1.5 Structure of the thesis

For a clear presentation of the concepts and the work, the paper is structured on five interconnected chapters.

Chapter 1 (Introduction) describes the context of the problem, outlines the relevance of ensuring correctness in database systems, and defines the motivation, objectives, and contributions of the current research.

Chapter 2 (Theoretical foundations) details the conceptual framework that is needed to understand the testing mechanisms. This cover the differential fuzzing principles, the way abstract syntax trees are generated and how the testing oracles work, presenting at the same time the state of the art of technology.

Chapter 3 (Application development) describes in details the software implementation phase of the orchestration platform. Architectural design decisions are exposed, the database structure used for history persistence and the technical asynchronous manipulation mechanisms' of Java sub-processes in the Python environment.

Chapter 4 (Experiments) present the testing methodology and empirical results obtained after developing the platform. The chapter includes the comparative analysis of the success rate and the throughput for the tested systems, discusses the nature of the identified 19 bugs and finalizes with an analysis of the threats to validity of the study.

Chapter 5 (Conclusions) summarizes the results of the entire research and engineering process, evaluating the degree of achievement of the objectives and propos-

ing directions for expanding the platform, such as full Docker containerization and the integration of additional query generators.

In conclusion, the main objective of this thesis is to address the technological gap between the capacity for automatically generating queries for database testing and the possibility of managing, visualizing, and analyzing these tests. Through designing a centralized orchestration web platform and applying this in a practical environment to discover bugs, the paper aims to demonstrate that introducing an observability layer can optimize the analysis process of database management quality.

1.6 Declaration of Generative AI and AI-assisted technologies in the writing process

Statement: During the preparation of this work, the author used Google Gemini to summarize papers and rephrase ideas for better understandability. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the thesis.

Chapter 2

Systematic literature review on Testing database applications

2.1 Systematic literature review (SLR). Overview

In this chapter, our aim is to present the existing knowledge about testing database applications. In the context of the thesis, we introduce this chapter to position our work with respect to existing research.

2.2 Study design

We characterize the state-of-the-art in the use of database applications following the systematic literature review (SLR) methodology [Kit04] [KC07]. The SLR guidelines for software engineering are provided by Kitchenham et al. [KC07], who propose the three main phases, summarized as follows:

1. An SLR starts with the planning phase, which identifies the need to conduct the SLR; then, the research questions that need to be answered are specified, and the rules for conducting the SLR are defined.

2. During the second phase, the conducting phase, the previous steps are followed, and primary data are extracted

3. The last phase is where all the information from the SLR, including the motivation, collection, and reporting of the review, is provided through a report.

Following these guidelines, we introduce the research process used in this study, the three phases described above are followed, each containing at least one or more activities.

2.2.1 Review need identification

Database systems [Wan20] represent the core pillar of the most modern software applications. At the current pace of increasing data volume and the transition to distributed systems architecture, ensuring the quality and performance of these systems has become a major challenge. Errors at the database level [LWX+25] can lead to financial losses [CSY+16], data corruption [ZSS+18], and security breaches [Tan24].

Although the process of developing software evolved by adopting object-relational mapping frameworks like Hibernate or Spring Data [GH19], specialty literature [CSJ+14] often indicates that those frameworks introduce weaker performance or hidden complexity because sometimes they generate inefficient queries that are hard to detect through the usual methods [YYS+18]. On top of that, the traditional testing techniques [SJR+15] have been proved to be insufficient in detecting logical errors in the database engines, where the system is not crashing but its returning incorrect data [RS20]. Developers struggle choosing between the automatic data generation, fuzzing techniques and differential testing [AKM18, ZCH+20, Mö22].

In this context, there is a need to synthesize and analyze the actual state of database testing techniques. This SLR investigates how to identify and classify the modern methods of ensuring correctness (automatic data generation or differential testing) and analyzes the performance challenges imposed by modern architecture and the usage of ORMs (Object-Relational Mapping). The end goal is to offer a high-level picture of the tools and methodologies that can increase the quality of database-intensive applications.

2.2.2 Research questions definition

The goal of the SLR is addressed through two research questions, each with a defined objective, as follows:

RQ1 - What are the most common techniques for automatic test data generation and bugs detection in database systems? Manual database testing is inefficient because of the highly complex schemes and the integrity constraints. This question tries to identify how test data can be automatically generated and how logic bugs can be detected, the errors where the database is not crashing but returns wrong results.

RQ2 - What are the test-specific challenges imposed by performance, usage of ORM frameworks and distributed architectures? Modern applications rely on abstraction layers and distributed architecture, therefore, the impact of ORMs over performance and the way abstraction can introduce performance anti-patterns is studied along with the challenges of testing in distributed systems, where latency and data consistency are critical, and don't appear in monolith databases.

2.2.3 Protocol definition

During this activity, we define the steps and rules for conducting the SLR. These steps are the following:

- the databases and search terms used to search for the primary studies,
- the criteria to include or exclude studies from the systematic review,
- the data extraction strategy on how to obtain the required information from each study,
- the synthesis of the extraction data and,
- the dissemination of the results.

These steps are detailed in the following sections.

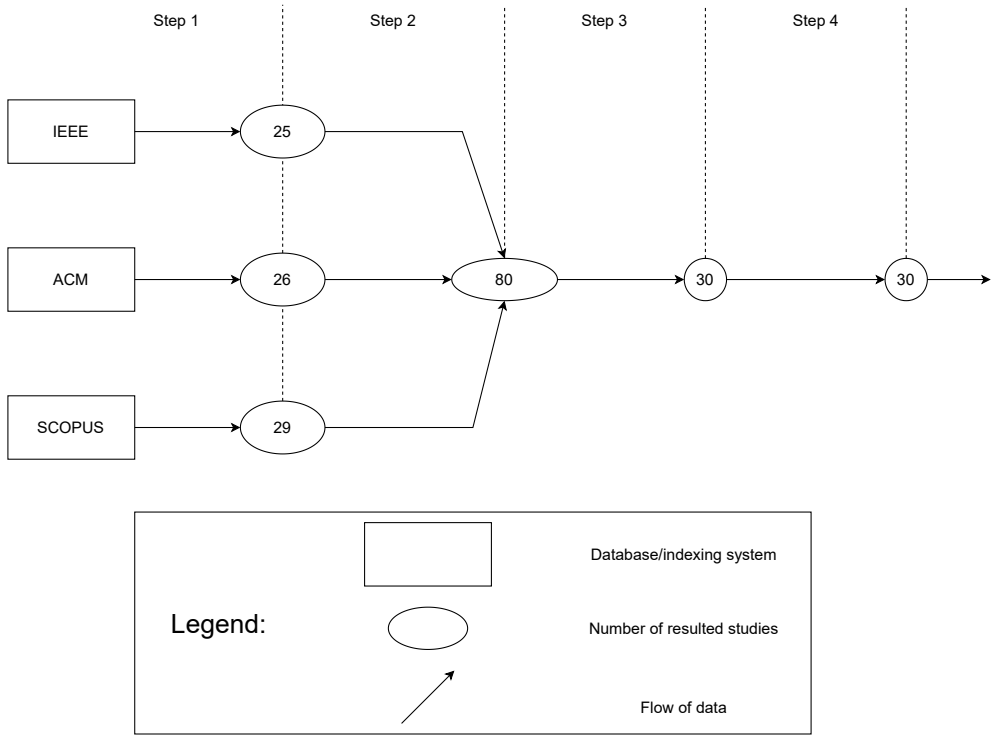


Figure 2.1: Systematic Literature Review Process Overview

2.3 Conducting the SLR

In order to conduct this study we followed the standard protocol for SLR, proposed by Kitchenham [KC07]. The process was structured in three phases: the planning the review, conducting it and disseminating the results. The objective of this section

is to detail the steps taken in order to do the SLR, ensuring the transparency and reproducibility of the selection process for the primary studies that lay at the base of this analysis.

2.3.1 Search and selection process

The goal of the search and selection phase is to find studies that address our research questions. We organize this phase into three steps, described below.

First, we define the databases and the keywords used to search for relevant studies. In the same first step, we start the search process in the databases using the keywords and search strings. The second step implies merging the results that were found in all of the previously defined databases into a single spread-sheet and the removal of the duplicates. The merged studies are filtered in the third step, using the previously defined inclusion and exclusion criteria.

The steps of this search and selection process are described in more detail in the next sections.

Database search

Firstly, we have chosen three different databases and indexing systems, such as IEEE Library, ACM Library and Scopus to search for relevant studies. The decision to use these sources was made because they are the most relevant for this study and highly accessible. They are reliable and contain a high number of articles in the computer science field. The mentioned resources are presented in the Table 2.1 below.

Name	Type	URL
IEEE Xplore	Electronic database	`https://ieeexplore.ieee.org`
ACM DL	Electronic Database	`https://dl.acm.org`
Scopus	Indexing system	`https://www.scopus.com`

Table 2.1: Databases and indexing systems used

Merging, and duplicates and impurity removal

The results from the previous step were added into a single spreadsheet, and the duplicate entries were removed (i.e. with the same title, authors, year, etc.). We even removed entries that were not research papers, such as prefaces, books, forewords, book presentations, etc. Furthermore, most of the papers that were published before 2018 were removed.

After the merging activity, a final list of 80 total papers was obtained.

Application of the selection criteria

The 80 unique studies obtained after the merging and cleaning step were then screened according to a predefined set of inclusion and exclusion criteria.

The inclusion criteria required that a study:

- targets relational, NoSQL or database-intensive applications,
- proposes, evaluates or compares a technique related to testing, validation or performance analysis of database systems,
- is a peer-reviewed publication (journal, conference or workshop),
- is written in English and the full-text is accessible.

The exclusion criteria removed studies that:

- focus only on database design, tuning or query optimization without any testing or validation component,
- are surveys, tutorials, editorials, posters, theses or book chapters without an empirical evaluation,
- are duplicates or extended versions of already included work,
- are clearly out of scope with respect to the research questions.

First, we performed a title and abstract screening, which reduced the set from 80 to 40 candidate studies. Next, we carried out a full text review of these 40 papers and excluded those that did not actually address testing of database systems or did not provide sufficient methodological detail. After this step, we obtained a final set of 29 primary studies, which are listed in Table 2.2.

2.3.2 Data extraction

From the selection process described above we obtained a final set of 29 primary studies, enumerated in Table 2.2. Each paper has a unique id (from P1 to P29). The data extraction activity was done manually and for each study we extracted the following information: bibliographic data (authors, title, year of publication and its venue), type of contribution (i.e. testing technique, tool, empirical study), targeted systems (DBMS, ORMs, database-intensive applications), the testing goal (correctness, performance or both), testing technique (i.e. differential testing, automatic data generation) and evaluation setup (benchmark, case studies, etc.). This structured extraction was used in the data synthesis to answer RQ1 and RQ2.

In line with the classification in [RB22], this review is designed as a scoping study, thus the objective is not to perform a quantitative meta-analysis, but to map existing approaches for testing database applications, summarize the main types of techniques and tools, and identify gaps and opportunities for future work.

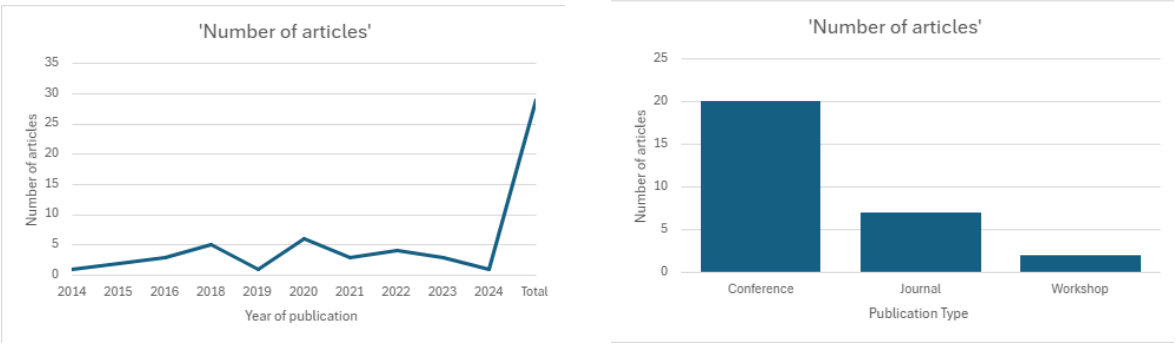
Table 2.2: List of the selected articles

Id	Citation	Title	Year
P1	[LWX+25]	Study of bugs in relational DBMS	2025
P2	[SCA+21]	Differential testing of ORM systems	2021
P3	[Wan20]	Analysis of database programming in software engineering	2020
P4	[CAD18]	Live data migration in distributed datastores	2018
P5	[YYS+18]	Performance bugs in DB-backed web applications	2018
P6	[Hon20]	Database testing in software development	2020
P7	[SV23]	DB testing techniques in web development	2023
P8	[LPR22]	Assertions vs differential testing in web testing	2022
P9	[BD21]	In-memory DB performance tests in Azure	2021
P10	[Tan24]	DB testing with data encryption	2024
P11	[CSJ+14]	Performance anti-patterns in ORM-based applications	2014
P12	[CSY+16]	Maintaining ORM code in Java systems	2014
P13	[SJR+15]	Effectiveness of automatically generated unit tests	2015
P14	[Sha15]	Automated unit test generation for evolving software	2015
P15	[CZPC21]	Comparison of REST API test generation tools	2021
P16	[DDP+23]	Class-level integration tests from call-site information	2023
P17	[GH19]	Spring reliability in DB bridging layer	2019
P18	[AAG+18]	Testing database applications with Polygraph	2018
P19	[AKM18]	DOMINO: test data generation for DB schemas	2018
P20	[MWK+16]	SchemaAnalyst: search-based DB test data generation	2016
P21	[WLM+21]	Analytical DB testing platform in telecom	2021
P22	[Mö22]	Overview of differential testing	2022
P23	[WLZ+23]	Query-aware database generator for evaluation	2023
P24	[RS20]	Finding DB bugs via query partitioning	2020
P25	[ZCH+20]	SQUIRREL: DBMS testing with feedback	2020
P26	[CKL+18]	Survey of metamorphic testing	2018
P27	[ZSS+18]	Transactions with inconsistent replication	2018
P28	[TSM+20]	CockroachDB: geo-distributed SQL database	2020

Id	Citation	Title	Year
P29	[LZAO22]	Performance bugs via equivalent queries	2022

2.3.3 Data synthesis

To answer RQ1 and RQ2, we classified the selected studies according to their main testing approach, tools or implementations they use, and the type of case study they report.



(a) Publication Trend (2014-2024)

(b) Publication Venue Type

Figure 2.2: General Statistics of the Selected Primary Studies

As illustrated in Figure 2.2a, the interest in database testing has shown fluctuations, with a notable peak of activity around 2020. Regarding the publication venues (Figure 2.2b), the majority of the selected studies were published in conference proceedings, followed by journals, indicating a dynamic research field focused on rapid dissemination of results.

Synthesis for RQ1: Functional Correctness

To address the first research question regarding correctness and bug detection, we analyzed the dominant testing techniques. Figure 2.3 presents the distribution of these approaches.

Automatic data generation is essential to ensure high SQL code coverage, thus the analyzed studies show that there is a competition between the search based and the constructive approaches.

The study was conducted by McMinn et al. [MWK+16] proves the efficiency of the search algorithms in data generation which satisfy complex integrity constraints, perhaps this method is computationally intensive. Alsharif et al. [AKM18] proposed a more constructive and deterministic approach, proving that using a constraint solver can generate test data faster, maintaining the same code coverage.

For the Big Data scenarios, Wang et al. [WLZ+23] introduced the query-aware concept, capable of synthesizing massive data volumes, which guarantees the return of some specific results, overcoming the classical random generators.

A discovery is the emerging of Metamorphic testing as a standard approach for logic bug detection (errors where the database is not crashing but returns wrong data).

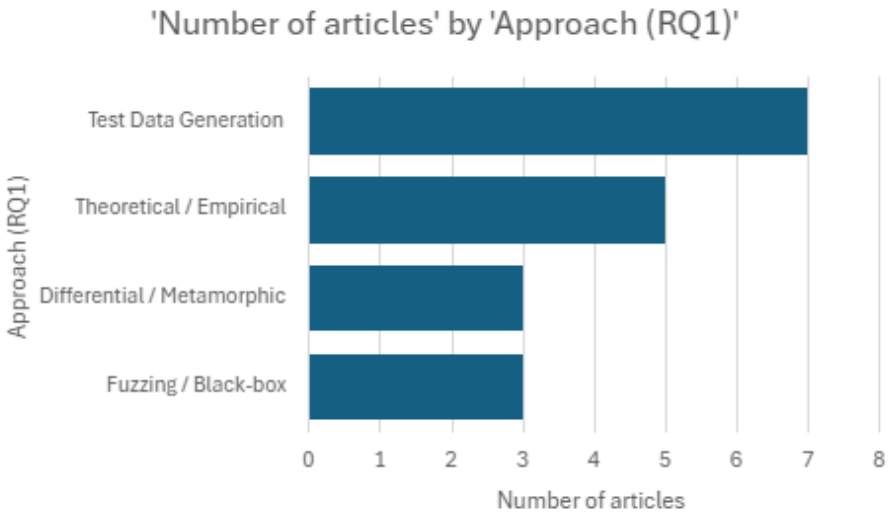


Figure 2.3: Distribution of Testing Techniques for RQ1 (Correctness)

The reference study of Rigger and Su [RS20], introduces the ternary logic partitioning technique, implemented in the SQLancer tool, which showed that decomposing an interrogation in multiple logical sub-parts, can detect lots of critical bugs in mature systems. Similarly, Zhong et al [ZCH+20] combines the syntactic validation with Fuzzing techniques successfully reaching the deep layers of the database engine, finding memory and security errors, which can't be found by the unit tests.

In the context of web apps, Leotta et al. [LPR22] compares the classical assertions with differential testing, summarizing that the latter is superior in detecting the visual and functional regressions. Möller et al. [Mö22] formalizes these concepts and explains how running the same input on different implementations can have an automatic oracle effect.

In spite of the technological advancements, the empirical studies like the one made by Shamshiri et al. [SJR+15], raises the limitations, because the automatic tools obtain a high code coverage, their capacity of catching real bugs is still moderate, emphasizing the necessity of human intervention when defining complex assertions.

Table 2.3: Summary of approaches for RQ1 (Correctness)

ID	Approach	Tool	Case study
P1	Taxonomy of failure types	–	Real-world bug reports
P2	Differential testing of ORM	Prototype framework	Java ORM frameworks
P3	Theoretical analysis	–	Conceptual analysis
P6	Theoretical analysis of DB testing	–	High-level discussion
P7	Web DB testing techniques	–	Web app context

ID	Approach	Tool	Case study
P8	Assertions vs. Differential	Test harness	Web applications
P10	Encrypted DB testing	Framework	Prototype encrypted DB
P13	Auto-generated unit tests	Multiple tools	Real-world projects
P14	Change-aware test generation	Tool	Evolving software
P15	Black-box REST API generation	Several tools	REST APIs
P16	Integration tests generation	Genetic algorithm	Java systems
P18	Transaction log analysis	Polygraph	Web applications
P19	Constructive data generation	DOMINO	Benchmarks
P20	Search-based data generation	SchemaAnalyst	Relational schemas
P21	Analytical testing platform	Platform	Telecom industry
P22	Differential testing overview	–	Conceptual examples
P23	Query-aware data generator	Generator	Synthetic databases
P24	Logic bug detection (TLP)	SQLancer	Multiple DBMSs
P25	Coverage-guided fuzzing	SQUIRREL	Multiple DBMSs
P26	Metamorphic testing review	–	Survey

Synthesis for RQ2: Performance and Architecture

Regarding the second research question, we focused on the performance implications and architectural challenges. Figure 2.4 highlights the primary focus areas found in the selected papers.

Analyzing studies related to performance (RQ2), we observed that a significant number of papers address the overhead introduced by Object-Relational Mapping (ORM) frameworks. As shown in Table 2.4, static analysis tools are frequently used to detect performance anti-patterns (P5, P11). Furthermore, in the context of distributed systems, the focus shifts towards maintaining data consistency and handling replication latency (P27, P28).

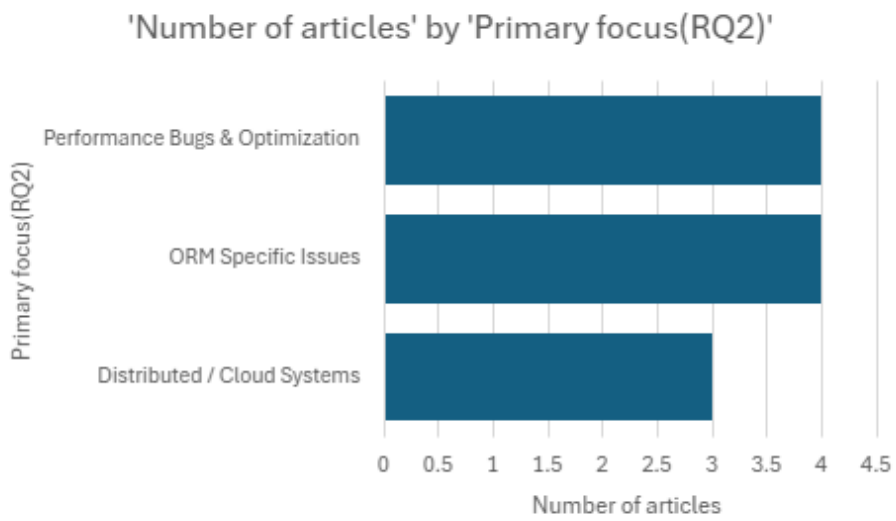


Figure 2.4: Distribution of Topics for RQ2 (Performance & Architecture)

Table 2.4: Summary of approaches for RQ2 (Performance)

ID	Approach	Tool	Case study
P4	Live data migration	Migration mechanism	Distributed datastores
P5	Performance bugs/anti-patterns	Analysis scripts	Open-source web apps
P9	In-memory DB testing	Scripts	Azure instances
P11	ORM performance anti-patterns	Static analysis tool	Java ORM apps
P12	Maintaining ORM code	Mining scripts	Java systems
P17	Spring reliability testing	Test harness	Java/Spring apps
P27	Inconsistent replication (TAPIR)	Protocol	Distributed storage
P28	Geo-distributed SQL	CockroachDB	SQL benchmarks
P29	Performance bugs (Metamorphic)	Prototype detector	Relational DBMSs

2.4 Results

This section answers the two research questions defined in the previous section. After synthesizing evidence from the 29 primary studies listed in Table 2.2 and summarized in Tables 2.3 and 2.4, our first objective was to present the evidence related to techniques for automatic test-data generation and bug detection (RQ1), then discuss findings related to performance, ORM usage, and distributed architecture (RQ2).

2.4.1 Answer to RQ1: What are the most common techniques for automatic test data generation and bugs detection in database systems?

The analysis of the literature reveals a clear transition from simple methods of generating test-data to more sophisticated approaches, capable of solving the oracle's problem in databases, as outlined in the above Section 2.3.3.

Regarding test data generation, synthesis shows a competition between search-based algorithms and constructive approaches. Although random methods are fast, recent studies (ex: Domino [AKM18], SchemaAnalyst [MWK+16]) show that using Constraint Solver is superior. These succeed in managing relational schemes with complex dependencies (foreign keys), guaranteeing a high code coverage and deterministic execution time, something that is impossible to achieve only through truly random data generation.

Regarding bugs detection, the main identified challenge is detecting the no-crash bugs, where the database returns incorrect data without generating a system error. The most efficient solution identified is differential testing [SCA+21] and metamorphic testing (especially ternary logic partitioning, implemented in SQLancer [RS20]). These techniques automatically build an oracle by comparing results between different database engines, or between semantically equivalent queries, thus identifying critical logic errors which traditional unit tests don't see.

2.4.2 Answer to RQ2: What are the test-specific challenges imposed by performance, usage of ORM frameworks and distributed architectures?

The above analysis of the literature, as provided in Section 2.3.3, outlines the fact that although abstractization and data distribution offer major benefits, they could also introduce significant non-functional risks.

The biggest challenge is the performance overhead of abstraction introduced by the object relational frameworks. Although tools like hibernate accelerate development, they hide the complexity of the generated SQL, leading to "performance anti-patterns" like the N + 1 select problem or excessive data extraction. The literature suggests that these problems are best identified by specialized static analysis [CSJ+14], [YYS+18], and not by dynamic testing, which usually occurs too late in the development cycle to allow significant refactorization.

For distributed architecture and cloud, the testing paradigm shifts from verifying the local correctness to ensuring the geo-replication consistency. Standard ACID properties are harder to guaranty in systems like CockroachDB or AzureSQL. So, an efficient testing needs protocols that are capable of simulating the partitioning of the network and inconsistent replication states (ex: Tapir [ZSS+18], imposing a transition from simple functional tests to more complex chaos engineering scenarios to validate data durability.

2.5 Future work and opportunities

2.5.1 Discussions of results

The results of the systematic literature review conducted outline a major change in the paradigm of database testing in the past decade. Initially, the research focused mainly on fuzzing based on syntax and manual data generation. However, my synthesis shows that currently the academic community migrated towards semantic-aware approaches. The adoption of constraint solvers and ternary partitioning logic shows that efficient testing requires a deep understanding of the database scheme and its logic, not just random data generation.

Another conclusion is the "Abstractization paradox" introduced by the ORM frameworks, although tools like Hibernate or Spring Data have the goal of simplifying the development, selected studies outline the fact that those frameworks obscure the performance bottlenecks. My analysis shows that traditional functional testing is insufficient in detecting these problems, but static analysis and query log mining are the only reliable methods for assessing the "N+1 select" problem and inefficient fetch strategies before they go into production.

However, distributed databases forced a reevaluation of the correctness concept. The transition from SQL monolithic databases to NewSQL geo-distributed systems shifted the testing objective from simply ensuring the ACID properties to validating the consistency models in network partitioning conditions. The literature analysis shows that chaos engineering and protocol conscious fuzzing are not optional anymore, as they are rather mandatory for these modern architectures.

2.5.2 Gaps, challenges, open issues

Despite the progress made, a major identified gap is the limited adoption of academic tools in industry. Although tools like SQLancer have discovered hundreds of bugs in popular database management systems, many other approaches remained at the prototype level, with limited documentation or support. Integration of these advanced fuzzers into the standard CI/CD pipelines remains a challenge due to the high computational cost and difficulty of configuring test oracles (maintaining multiple database engines for differential testing).

Second of all, there is a notable lack of research towards NoSQL and multi-model databases. The vast majority of the 29 primary studies focus on SQL relational systems. As the industry heads towards polyglot persistency (using Redis, MongoDB and Neo4j in the same architecture), the lack of unified testing in order to validate the data consistency between storage engines represents an open significant problem.

Not least, the "Oracle problem" is still only partially solved. Although metamorphic and differential testing offers automatic ways to find errors, they often generate false positives or require significant manual effort for triage. Distinction between a legitimate functionality difference (for example, the precision of the floating comma between MySQL and

PostgreSQL) and a real logic bug remains a high time consuming challenge for developers.

2.5.3 Opportunities/Recommendations

Integration of Large Language Models (LLMs) in database testing is a promising direction for future works. While current fuzzers are based on predefined grammars, LLMs could be trained to generate semantically valid SQL queries that focus on specific edge cases or on business logic, potentially overcoming the capped coverage percentage of the traditional fuzzers. Also, researchers should focus on developing some easy to use tools for validating cloud-native databases, especially for serverless environments where traditional profiling of performance is difficult.

Based on the performance risks identified in the ORMs, the recommendation is to adopt the continuous performance strategy testing. Teams that use ORMs should integrate static analysis tools (like the ones identified in studies P5 and P11) in their build process, in order to signal anti-patterns in advance. For distributed systems, practitioners are encouraged to adopt chaos engineering principles, injecting network latency and node errors into staging environments in order to verify the resilience of the system, instead of relying exclusively on ”happy path” functional tests.

There is a clear opportunity for reducing the gaps between the academic prototypes and the industry’s needs by developing some ”Test-as-a-service”-like tools. Instead of having developers configure locally complex constraint solvers, future tools could offer containerized environments, pre-configured, which automatically generate schema-aware test data, thus facilitating the advanced database testing techniques.

Chapter 3

Orchestration platform for SQLancer

This chapter details the process of developing and implementing the practical application that underlies this dissertation work: a full-stack web platform designed to orchestrate, monitor, and analyze the testing campaigns using the SQLancer tool. The main scope of this application is to transition from command-based execution of the testing tool to a centralized, intuitive, and scalable environment. The application automates the initiation of experiments, retrieves execution logs in real time, and stores the results, simultaneously offering an advanced dashboard for visualizing the performance of different database management systems. In the following sections, the architectural motivation, key implemented features, as well as the tech stack that ensures the friendly interface and robust backend will be detailed.

3.1 Problem statement and background

In modern software engineering, the performance analysis, availability, and health of complex systems are based on consolidated observability paradigms. The core pillars are metrics collection, logs, and tracing. The current technological ecosystem is dominated by open-source reference solutions, such as Prometheus and Grafana, which are used on a large scale in production environments for critical infrastructure monitoring and databases.

Prometheus acts like a time-series database, using a metrics collecting model that is pull-based. Every once in a while, it queries specific endpoints that are also called exporters in order to collect numerical values associated with a timestamp. Prometheus excels in collecting system metrics, such as CPU usage, RAM consumption, I/O operations, and operational metrics of databases, including active connections, the size of the buffer pool, cache's success rate, and transactions per second. On the other hand, Grafana has become the standard in the industry for data visualization and creating interactive dashboards. This connects to multiple data sources and transforms complex queries into advanced graphical representations and alerting systems based on predefined thresholds.

With all this, although Prometheus and Grafana are remarkable tools for operational monitoring, applying them directly in the context of automated logical testing of databases

through fuzzing techniques, highlights conceptual and architectural limitations:

- **Quantitative vs semantic metrics:** Traditional monitoring systems measure the physical state of the application, not the logical correctness of its results. If a database is tested with SQLancer and, because of a bug in the query optimizer, returns an incorrect dataset for a complex WHERE clause, the infrastructure will seem perfectly functional, the server reports a successful execution code, the resource consumption remains normal, and the latency is optimal. For Prometheus and Grafana, this event is registered as a successful operation, ignoring the logical error. Differential fuzzing needs a tool that is capable of comparing logical result sets, something that is completely outside the scope of a database for time series.
- **Absence of process orchestration:** Prometheus and Grafana are passive observation components; they cannot initiate, parameterize, or stop a sub-process of the operating system. SQLancer executes jar files using the command line; this requires dynamic argument management.
- **Inadequate log management:** Although there are extensions like Grafana Loki for log management, they are optimized for structured log systems. Logs generated by a fuzzing engine are massive, unstructured, and can contain dumped SQL code, syntax errors, and Java stack traces.

Considering this structural deficiencies, the decision of developing a dedicated full-stack web application for a replication study is imperative.

3.1.1 Context of ensuring quality in database management systems

Database management systems represent the core pillar of modern digital infrastructure, being responsible for storing, handling, and recovering data in most computer software, from critical banking systems all the way to social media platforms. With this in mind, a logical error in query processing can lead to data corruption, incorrect results in financial reports, or security breaches.

Traditionally, testing these complex systems was based on manually written unit and integration tests. Although the declarative nature of SQL and the complexity of modern query optimizers make covering all scenarios through manual methods almost impossible. The automated fuzzing concept intervenes here, where tools like SQLancer have revolutionized logical bug detection through the automatic generation of huge volumes of interrogations and verifying the results with testing oracles (i.e. TLP, NoREC).

3.1.2 Problem identification: limitations of CLI based execution

Though SQLancer is a powerful tool capable of finding bugs in mature systems like SQLite, MySQL, or PostgreSQL, using it in the context of a development or research process can come with a few barriers:

1. **Lack of history persistence:** In its native form, SQLancer runs on the command line, so the results, throughput statistics, and error logs are shown in the terminal or saved as text files. Without a centralized database that saves each execution, it is impossible for the researchers to analyze the evolution of a database management system's reliance over the course of multiple iterations or versions.

2. **Difficulty of comparative analysis:** The behavior of a replication study on multiple database engines (i.e. comparing the efficiency of DuckDB and SQLite with the same testing oracle) implies manually running dozens of commands and individual data extraction. There is no automated method to present this data side by side in a visually intelligible format.

3. **Inexistent real time monitoring:** A testing session can take hours or even days, so the user doesn't have a simple way to see the live progress, query success rate, or determine whether the testing process has been stuck in a fatal system error without manually inspecting the background processes.

4. **Complexity of configuration:** Test parameterization using console flags is prone to human error and makes the process of experiment replication harder for other researchers.

3.1.3 Problem statement

The core problem addressed by this thesis is the lack of integrated infrastructure for visualizing and managing the automated testing processes of databases. Currently, there is a technological gap between the capacity for test generation and the interpretation, monitoring, and archiving of these tests in an efficient manner.

This thesis aims to solve this decay by developing an orchestration platform that can transition the manual database testing from a manual task, isolated, inside an automated continuous flux, monitored and visualized.

3.1.4 The objectives of the proposed solution

In order to answer the identified problems, the developed application aims to achieve the following specific objectives:

- **Data centralization:** Storing all the results of the experiments inside a relational database for later queries and auditing.

- **Live monitoring:** Offering a web interface which allows real time visualization of the execution status.

- **Dashboard:** Using data visualization techniques to highlight bug distribution and the throughput of each tested database management system.

- **Simplify operations:** Creating an abstraction layer over the Java CLI, allowing experiments to be run through intuitive web forms.

3.2 System design and architecture

This application's design was guided by the necessity of turning a powerful but hard to manage tool, into an accessible platform, scalable and easy to monitor. To reach this objective, the system was designed on the basis of separation of concerns principle, decoupling completely the interface with the user of the complex logic query execution of tests and data persistence.

3.2.1 Features

The app was developed while keeping in mind three major functional directions, each addressing a specific limitation of the traditional command line execution:

1. **Asynchronous experiment orchestration**: The most important feature of the system is the capacity to launch fuzzing sessions directly from an intuitive web interface. The user can pick the desired database management system, the compatible testing oracle and the time length of the experiment through some dynamic forms. Once started, the system doesn't block the user's interface, but delegates the task to a background process, this design prevents the bottlenecks at the web service level and allows the running of long experiments without the risk of interrupting the HTTP connection.

2. **Real time monitoring and log capture:** During an experiment, visibility is essential. The platform offers a dedicated section where the users can view the current state of all experiments (i.e. Running, Completed, Error, Timeout).

3. **Visual analytics and data aggregation:** To make sense of the large data volume generated from the tests, the application disposes an metric aggregation and normalization module. The system's dashboard takes raw data (number of queries, throughput, discovered bugs) and displays them as advanced graphical visualization. Functionality has standardization algorithms for naming, to avoid duplicate entries and automatic bug distribution on categories. This visual reporting feature transforms raw data into actionable insights, facilitating comparative studies between different database management systems.

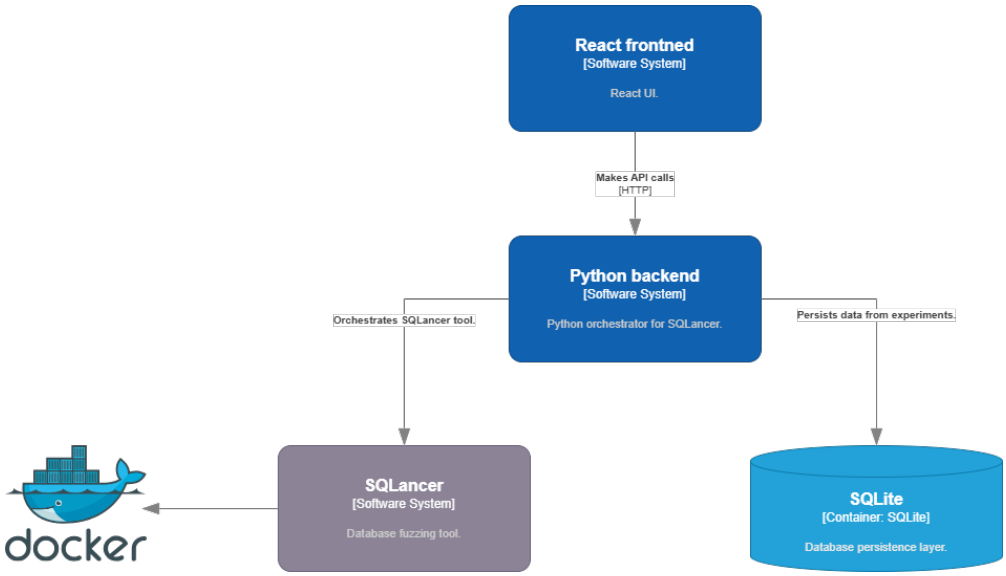


Figure 3.1: The architectural blueprint of the proposed full-stack fuzzing orchestration platform.

To provide a high level overview of the developed application, the structural organization of the platform is formally mapped in Figure 3.1. The diagram illustrates the complete flow of data and the interaction boundaries between the three primary architectural layers, as well as the mechanism used to manage the external testing lifecycle. As shown, the decoupled nature of the design ensures that the users interactions on the presentation layer do not interfere withe the resource-intensive tasks managed by the application layer.

3.2.2 System architecture

In order to sustain the feature described above, the platform was modeled using a classic three tier client-server architecture. This structure ensures a high modularity, allowing testing, maintenance and independent scaling of each component.

1. **Front-end - the presentation layer:** The user interface represent the main interaction point. It's built as a single page app, having the role of dynamic rendering the data without loading the whole web page, this level is responsible exclusively for display logic: send asynchronous requests to the backend level, manage application's states (loading, success, error) and define graphic elements (tables, graphics, forms). Through externalization of this task to the clients browser, resources of the server are saved for executing fuzzing processes.

2. **Backend - the application layer:** This level represents the brain of the system. The backend acts as a middleman between the users interaction and the operating system. It exposes a RESTful API which takes the front-end requests. A critical architectural task of this level is managing the sub processes: when it receives the run command of an experiment, the backend dynamically assembles the command line string needed

for the SQLancer tool, launches the Java process through the operating system's kernel and attaches listeners to capture the stdout and stderr data fluxes.

3. **Database - the data layer:** The relational database represents the foundation of information persistence in the proposed architecture. Unlike a simple text file save, the database imposes a strict schema, validating the integrity constraints. This level retains an immutable history of all tests: the unique identifier of the test, creation timestamp, the targeted engine, chosen configuration, as well as the terminal logs at the moment of finalizing. The relational structure is vital for the dashboards performance, allowing the backend to execute aggregation type queries (group by, sum) directly at the database level, before sending the results for visualization.

Through this decoupled architecture, the application transcends the status of a simple graphical interface, becoming a robust orchestration platform, capable of efficiently managing the complete life cycle of an automated database testing experiment.

3.3 Implementation

This chapter details the practical process of transposing the proposed architecture into a functional software solution. The implementation assumed the integration of a set of modern web technologies with operating system level processes, with the goal of building a robust, asynchronous, and error resilient platform.

3.3.1 Technologies and frameworks

The selection of the tech stack was made based on performance, asynchronicity and ease of development requirements:

- **Front-end:** React, a declarative JavaScript library, was chosen due to it reusable components architecture, ideal for building a complex dashboard. For the state management and efficient query of the API TanStack Query (React Query) was chosen, a tool which automates background data fetching and implements polling strategies. Styling was done using Tailwind CSS, a utility first framework that enabled the creation of a modern and responsive design, with specific accents for technical terminals, without writing external CSS files that are hard to maintain. The generation of charts has been delegated to the Recharts library
- **Backend:** It was developed in Python, using the FastAPI framework. Python was chosen due to it excellent support for manipulating system sub-processes through standard libraries and its rich ecosystem for text parsing.
- **Database:** For data persistency SQLite was used. Although it's a lightweight and serverless database, it was considered the optimal choice because the application doesn't need massive concurrency for writing from thousands of users, but it serves

as an administrative research tool, offering at the same time safe transactions and maximum portability for the data file.

3.3.2 Database schema and data persistence

Unlike a machine learning model that uses a static dataset, this system generates and stores data dynamically over its life-cycle. Database schema was designed to normalize the extracted information from the testing executions.

The central component of this schema is the table dedicated to experiments, which acts as a complete registry of system's activity. For each launched fuzzing session, critical metadata is being stored: unique identifier, current state of the process, configuration parameters, and timestamps. Special attention was paid to storing testing artifacts; the raw-output column was defined to store big blocks of unstructured text, faithfully capturing the standard output of the SQLancer tool. This approach ensures the traceability and reproducibility of each experiment.

3.3.3 Process orchestration and SQLancer integration

The major technical challenge of this implementation was the orchestration of a command line based tool, written in Java, from a web server based on Python, in a non blocking execution manner.

Listing 3.1: Main async orchestration for the SQLancer process at operating system level.

```
async def run_sqlancer(exp_id: str, config: ExperimentConfig):
    process = subprocess.Popen(
        cmd,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True, bufsize=1, universal_newlines=True
    )

    start_time = time.time()

    t_out = threading.Thread(target=read_output, args=(process.stdout,
        stdout_lines, False))
    t_err = threading.Thread(target=read_output, args=(process.stderr,
        stderr_lines, True))
    t_out.start()
    t_err.start()

    while True:
        if process.poll() is not None:
            break

```

```
20          elapsed_time = time.time() - start_time
21          if elapsed_time > config.duration_seconds:
22              if platform.system() == "Windows":
23                  subprocess.run(["taskkill", "/F", "/T", "/PID", str(
                        process.pid)],
24                                 stdout=subprocess.DEVNULL, stderr=
                                       subprocess.DEVNULL)
25              else:
26                  process.terminate()
27              break
28
29          time.sleep(1)
30
31      t_out.join()
32      t_err.join()
```

The above listing 3.1 showcases the core of the orchestration module from the backend. Synchronous functions like subprocess.run were avoided intentionally. This architectural decision allows launching the Java tool in a child process of its own, without blocking the event loop of the main web server.

A critical aspect of this implementation is represented by the management of output flows. Because the operating system allocates buffers that are limited when communicating between pipes, a high volume of logs generated by SQLancer could fill this buffer, causing a deadlock. In order to prevent this scenario, the platform instantiates two independent threads that consume the data asynchronous and continuously.

In order to ensure the stability of the platform against the fuzzing processes that enter in infinite loops or refuse to stop, an active monitoring loop has been implemented. This periodically checks the processes status (*process.poll()*) and, when the allocated time limit has been passed (*config.duration_seconds*), it intervenes at the kernel level to send forced termination signals (using operating system-specific commands like *taskkill* on Windows).

When a user initiates an experiment from the web interface, the backend dynamically assembles an operating system command. Instead of using synchronous execution functions which would block the HTTP server until the test finishes (which could take hours), the system uses asynchronous delegation (through sub-process specific modules from Python).

The system injects the process in the background and monitors its process id. At regular intervals, or upon process completion, the server reads the output streams of the Java instance, sanitizes them, and writes them in the SQLite database. Using this asynchronous architectural design, the web platform remains always responsive, allowing the user to navigate through the interface and follow the evolution of the test.

3.3.4 Telemetry, metrics, and visualization

The efficiency of a logical testing campaign for databases does not measure in classic metrics used int artificial intelligence (precision, recall), but through system telemetry that has been

generated during the fuzzing. The implementation integrated mechanisms for extracting four fundamental performance metrics:

1. **Total queries:** The gross number of generated SQL queries and sent to the database management system.
2. **Throughput (queries per second, QPS):** Rate of queries per second, a crucial indicator of the way in which the database engine can handle continuous stress.
3. **Success rate:** Percentage of queries that did not end with syntax errors or crashes.
4. **Logical bugs found:** The number of discrepancies identified by the testing oracle.

To make sense of these raw metrics, a visual aggregation component was implemented. Before displaying, the data goes through a normalization method written in React, which eliminates the semantic duplicates generated by the CLI inconsistencies (automatic combination of ”SQLite WHERE” with ”sqlite3 TLP-WHERE” labels). Subsequently, the data is transposed into pie charts and bar charts, providing an imminent graphical representation of the comparative performance between systems like DuckDB, SQLite or PostgreSQL.

Listing 3.2: Normalization and standardization method for oracles at the fron-end level.

```
function normalizeOracle(name: string) {
  if (!name) return "Unknown";

  let clean = name.replace(/_/g, " ").toUpperCase().trim();

  if (clean === "WHERE") return "TLP WHERE";
  if (clean === "HAVING") return "TLP HAVING";
  if (clean === "GROUP BY") return "TLP GROUP BY";
  if (clean === "AGGREGATE") return "TLP AGGREGATE";
  if (clean === "DISTINCT") return "TLP DISTINCT";

  if (clean === "NOREC") return "NoREC";
  if (clean.includes("QUERY PART")) return "QUERY PART.";

  return clean;
}
```

To ensure the accuracy of data aggregation in the dashboard, it was necessary to implement a normalization mechanism for the data, at the client level interface. As it is show in 3.2, the normalizeOracle method acts like a cleaning and standardization filter for the metadata received from the backend.

This processing is vital because the entries generated by the command line execution of the SQLancer tool, often contain syntactic inconsequences (using the character underscore instead of the whitespace in TLP WHERE), and the data taken from the static replication studies often use short forms (like WHERE instead of TLP WHERE). The method uses

regular expressions remove unwanted characters, standardizes the text by converting it to uppercase, and dynamically maps abbreviated variants to their formal identifiers. In the absence of this algorithmic normalization step, the charting engine would interpret these syntactic variations as distinct entities, leading to duplicate bars on the throughput axis and visually distorting the results of the comparative analysis.

Chapter 4

Experiments

This chapter provides an in depth view of the practical evaluation of the developed orchestration platform and the results of the automated testing campaigns that have been conducted on multiple database management systems. The main objectives of these experiments are: on one hand, to validate the functionality of the developed web platform (proving its capacity to run and extract real data from real fuzzing sessions), and on the other hand, to conduct a replication study and an extension of the original SQLancer paper.

Using the experiments described below, we examine the robustness of some mature database engines towards well known testing oracles (like TLP, NoREC, and PQS). Also, this chapter gives details about the testing methodology, collected metrics, and a deep analysis of the 19 logical bugs identified during the runs, ending with an objective discussion about the threats to the validity of the study.

4.1 Experimental setup and target systems

Unlike the traditional empirical studies or the automated learning (machine learning), database testing through differential fuzzing is not based on a static dataset. Thus, the SQLancer tool dynamically generates database schema (tables, indexes, views), as well as massive volumes of SQL queries, keeping in mind each target engine's specific dialect. So, the base of this experiment is made of a rigorous selection of some database management systems alongside with their versions.

In order to ensure the relevance of the study, a hybrid selection of target systems was chosen, divided in two main categories:

1. **Replication targets:** Database management systems that have been tested in the original SQLancer paper, selected in order to verify if vulnerabilities discovered in 2020 have been solved and if there are any regressions in the new versions. SQLite, MySQL, PostgreSQL, CockroachDB and DuckDB are part of this category.

2. **Extension targets:** Emergent database management engines or intensely used in the industry, that have not been exhaustively covered in the anterior studies, or they ben-

efit from experimental support int SQLancer. Here TiDB (a NewSQL distributed system) and MariaDB (a popular fork of MySQL) have been included.

System's versions and instrumentation particularities

The testing campaign targeted modern versions of these systems, in order to evaluate the current stage of their robustness. The following list summarizes the target configuration:

- **SQLite (v3.49.1):** Being on of the most tested systems in the world, it served as a baseline for evaluation of the TLP oracles (WHERE, GROUP BY, AGGREGATE), NoREC and PQS.
- **MySQL (v8.0.46):** Tested to evaluate its complex optimizer on the TLP WHERE and PQS oracles (the NoREC support not being available in the used SQLancer version).
- **TiDB:** Selected for its distributed architecture, proved to be the most productive testing environment in this experiment, especially when applying the TLP HAVING oracle, which has not been documented in the original literature for this system..
- **MariaDB:** Introduced as an addition to the original work, successfully ran the NoREC and DQP oracles.
- **Incompatible systems:** A reality about the replication studies is represented by the degradation of the software tools in time. During the configuration, PostgreSQL (v15.17) and CockroachDB (v13.0.0) have raised severe compatibility problems. SQLancer 2.0.0 generated internal exceptions like *NullPointerException* while interacting with PG 15.x, and the instrumentation settings (sql.metrics.statement.details.plan.collection.enabled) needed for CockroachDB have been removed by the developer in version 13. So, these two systems could not generate valid results, which has been documented as a limitation of the study.

This diversity of the testing environment—containing integrated SQL systems, traditional client-server systems, and distributed systems—provides a good starting point for a comprehensive analysis of the efficiency of modern techniques for finding logical bugs.

4.2 Design of Experiments

The design of the testing campaign is based on the reproducibility and automation principles. Unlike the original SQLancer study, in which the experiments needed manual interventions and bash scripts, the presented methodology used the orchestration platform developed in Chapter 3. This approach ensured a strict standardization on the testing environment and a uniform metrics collection.

4.2.1 Parametrization and time constraints

By its nature, fuzzing is a probability based process, which theoretically can run to infinity. In the academic research environments, test campaigns can stretch over the course of few days or even weeks in order to maximize the chances of discovering edge case bugs.

For this study, because of time constraints and the limited computational resources and in order to prove the efficiency of the platform in a fast development cycle, a short-burst fuzzing testing strategy was adopted. Each testing session was limited to a duration of 5 to 10 minutes per oracle. Although this time window is reduced compared to specialty literature, the objective wasn't the exhaustion of the database management systems search space, but proving the fact that using a modern configuration and an orchestration platform, the critical logic bugs can be discovered even in the first minutes of execution. All the experiments were configured to use a high concurrency level, allowing the generation of a massive throughput in this limited time frame.

4.2.2 Automated execution flow

Each experimental iteration followed a standardized workflow, orchestrated directly from the web platform's graphical interface:

1. **Initialization:** The user selects the target database engine and the desired oracle from the application's form.
2. **Environment generation:** SQLancer dynamically creates a new database, populates it with tables, indexes, views, and inserts rows with pseudo-random data, generating the initial database state.
3. **Oracles execution:** The generation and verifying of the queries is launched. For example, in the TLP oracles case, the system creates an original query and three partitioned subqueries, comparing later the results.
4. **Monitoring and interception:** The platform captures the *stdout* and *stderr* errors. When SQLancer identifies a discrepancy (potential bug), the database state and query that produced the error are extracted automatically.
5. **Final aggregation:** When reaching the timeout, the platform stops the Java process, parses the logs in order to extract the total number of queries and the success rate, inserting a complete, immutable report, in the application's SQLite database.

4.2.3 Mapping the oracles to the target systems

A critical aspect of the experimental design is that not all the testing oracles are compatible with all the database management systems. This limitation arises from the development stage of the SQLancer tool and the uneven support for some SQL features.

The testing matrix was adapted as follows:

- **SQLite** benefits from the most exhaustive testing, supporting the TLP evaluation (WHERE, GROUP BY, AGGREGATE clauses), NoREC, and PQS.
- **MySQL** was tested mostly with TLP WHERE and PQS because there is no support for NoREC in the current version.
- **TiDB** was evaluated using TLP WHERE, HAVING, and QUERY-PARTITIONING, focusing on the HAVING clause, an area that has not been detailed in the original SQLancer study for this distributed system. Note: Rigger and Su (2020) included TiDB in their evaluation (latest as of May 2020) but implemented additional oracles beyond WHERE only for DBMSs whose WHERE oracle saturated; they did not implement the HAVING oracle for TiDB in the reported experiments. This thesis runs the TLP HAVING oracle on a newer TiDB build (v7.5.1) and reports the results here.
- **MariaDB** was a tool that exclusively had NoREC and DQP oracles available for it.

This asymmetric design, documented through the orchestration platform, reflects the practical limitations of testing in the real world and outlines the necessity of a platform that keeps precise track of which oracle ran on which system.

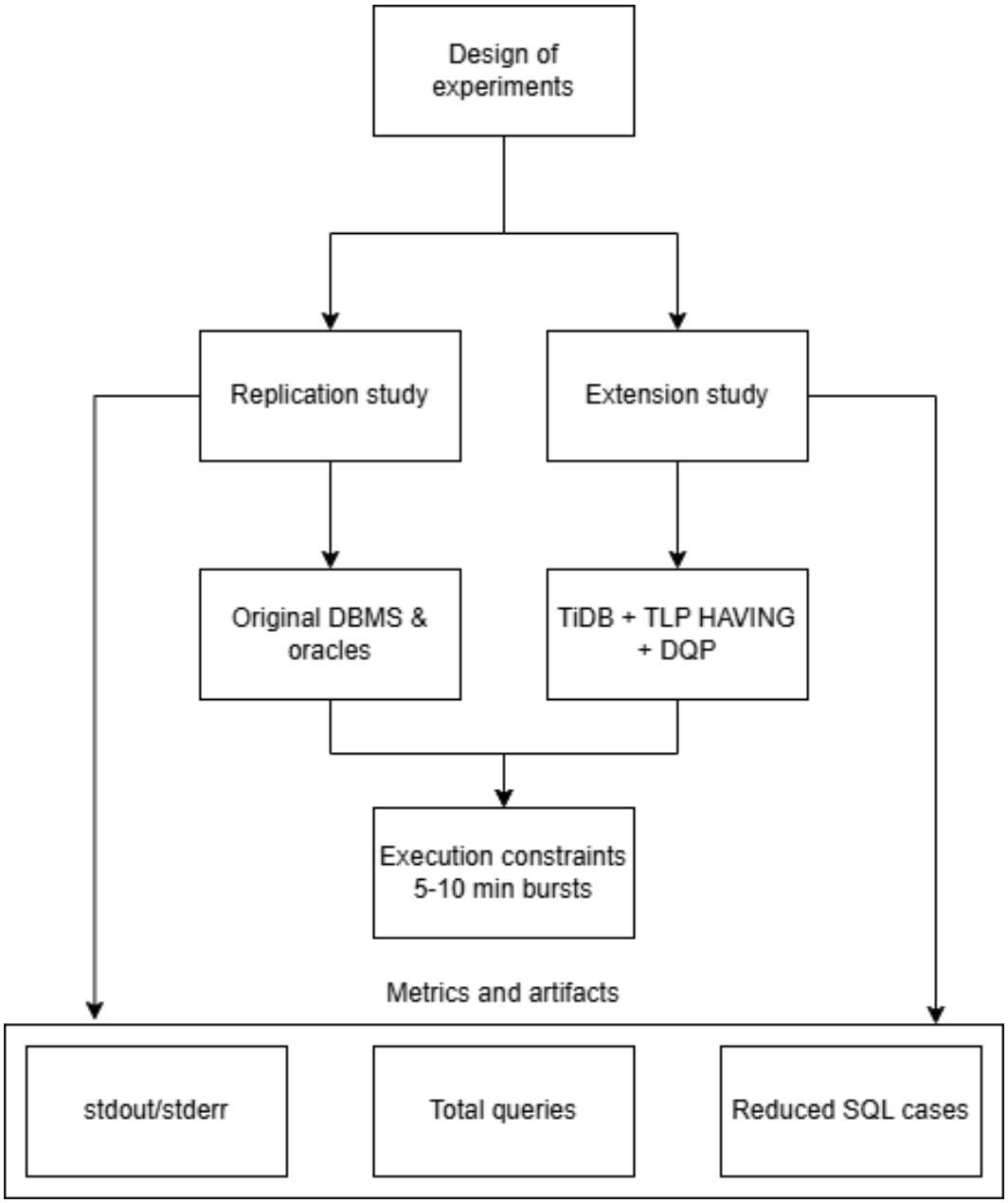


Figure 4.1: The hierarchical breakdown of the experimental design, highlighting the replication parameters and the extended study on TiDB.

4.3 Analysis metrics

Evaluating the effectiveness of an automated testing campaign requires close monitoring of the telemetry generated during the execution. Through the orchestration platform that I developed, the raw logs generated by SQLancer have been parsed asynchronously, allowing the extraction and storing of some identified standardized performance metrics. These metrics not only quantify the success of the bug discovery process, but also offers a clear image over the compatibility grade between the testing tool and the each database engine SQL dialect.

For the current analysis, there are four fundamental metrics that have been defined and collected:

1. **Throughput - queries per second**

   The throughput, measured in queries per second (q/s), represents the raw speed at which the system generates, sends, and validates the SQL queries. In the context of fuzzing, high throughput is vital: the more queries a system can evaluate in a short time, the more state space it explores, increasing the likelihood of reaching edge cases. Based on the experimental data collected, there are major variations in this metric depending on the architecture of the database management system and oracle used. For example, the compact systems that run in memory or with minimal network overhead, like SQLite and MariaDB, have shown excellent throughput (between 12,000 and 16,600 q/s on the TLP and NoREC oracles). On the opposite side, the oracles that require complex processing on huge datasets (like TLP AGGREGATE or distributed systems like TiDB) have registered considerably lower rates (1,100-3,500 q/s).

2. **Success rate**

   The success rate means the percentage of queries generated by SQLancer that have been accepted and successfully executed by the database management engine, without triggering syntax errors or routine semantic exceptions (divide by zero, type mismatch). This metric is a direct indicator of the generator's quality abstract syntax tree from SQLancer: a high success rate proves a precise understanding of the SQL dialect. The experiments show that implementations like MySQL and DuckDB are very mature in SQLancer, reaching a 99 percent success rate. On the other hand, for specific oracles like TLP HAVING running on TiDB, the success rate was only 53 percent, or 55 percent for TLP AGGREGATE on SQLite, pointing out that the generator frequently builds SQL structures that are rejected by the database parser.

3. **Total queries executed**

   This metric represents the absolute quantity of processed queries in a testing session and is directly proportional to the length of the experiment and the sustained throughput. The correlation of this number with the number of discovered bugs allows us to compute the cost of discovering a vulnerability. In the following experiments, although the sessions were limited to 5-10 minute time windows, the total volume of queries was in the millions, proving the exceptional scalability of automated comparative testing.

4. **Logical bugs found**

   Unlike conventional software testing, where a bug manifests mostly through a crash, testing database engines through oracles (TLP, NoREC) reveals a logic bug, which is more subtle. This appears when the database engine works apparently perfectly but returns incorrect results. The oracles report a logic bug when they detect a discrepancy

between the result of the original query and the evaluation of the verification methods. This metric represents the end scope of the current research. Depending on the nature of the discrepancy, the platform classified the bugs as:

- **Size mismatch:** The engine returns a different number of rows.
- **Content mismatch:** The engine returns the same number of rows, but the values are computed incorrectly (frequently found in aggregation methods like MAX or SUM).

By aggregating these four metrics in the dashboard of the orchestration platform, it was possible not only to quantify the errors but also to create a performance profile for each database engine.

4.4 Results

In spite of the short time windows for the test executions, the orchestration platform managed to identify a total of 19 unique logic bugs distributed unevenly between the target tested systems. This result confirms the efficiency of the differential fuzzing testing and outlines the fact that the increasing complexity of the query optimizers, the risk of introducing subtle semantic defects remains high.

In the next sections the results for each database management system are detailed, analyzing the type of the queries that triggered errors.

4.4.1 SQLite results (v3.49.1)

Being the database engine with the highest oracle coverage in this study, SQLite showed remarkable robustness, but it was no immune to logical bugs. A total of 2 bugs were found:

- **Bug in the WHERE clause (size mismatch):** The first defect was triggered by the TLP WHERE oracle on a query that implied a view. The original query (SELECT ALL v0.c0 FROM v0) returned one row, while the TLP (which added BETWEEN clauses combined with the QUOTE() method and logic operators OR/AND) returned zero rows, losing one record. This type of error indicates a problem in the way the SQLite optimizer evaluates complex predicates that contain implicit conversions on views.
- **Bug in the AGGREGATE clause (content mismatch):** The second defect was identified by the TLP AGGREGATE oracle and implied the generation of an incorrect value (different from the one expected). The error appeared while evaluating the MAX(CHAR(...)) method on a FTS5 type view, using the UTF-16 encoding and a NOCASE collation rule. It is interesting that this pattern is very much alike a documented defect in the original SQLancer paper, proving that the interaction between the aggregation methods, data types and full-text search modules remain a vulnerable zone in the SQLite ecosistem.

4.4.2 MySQL results (v8.0.46)

Although MySQL registered a nearly perfect success rate, the TLP WHERE oracle managed to identify 2 logic bugs related strictly to predicates processing (both of size mismatch type):

- **Boolean concatenation error:** The first defect returned 5 rows instead of the 6 correct. This happened because of using the concatenation operator of character arrays (——) forced to be evaluated as a boolean predicate, combined with the IS UNKNOWN clause and the GREATEST method. MySQL optimizer wrongly filtered a valid row.
- **Error in flow control functions:** The second defect provided more rows that it should've (16 rows partitioned instead of the 14 from the initial query). The error was triggered by the usage of the IF(NULL, ...) method inside an equality predicate. This behavior shows a broken evaluation of the three valued logic - TRUE, FALSE, NULL, specific to the SQL language in the MySQL optimizer.

4.4.3 Results for TiDB

The distributed system TiDB proved to be the most productive environment in the testing campaign, generating 15 logic bugs. The most valuable aspect of this discovery is that this study exercised the TLP HAVING oracle on a newer TiDB build (v7.5.1); prior work (Rigger and Su, 2020) evaluated TiDB as of May 2020 but did not run the HAVING oracle for TiDB in their reported experiments. From the 15 defects:

- **13 bugs were triggered by TLP HAVING.** The structural analysis of these queries revealed a clear pattern: almost all the failed queries implied the usage of a NATURAL RIGHT JOIN combined with GROUP BY and HAVING, having attached specific hints for the optimizer (/*+ NO INDEX MERGE()*/ or /*+INL JOIN()*/). 9 of these errors were of the content mismatch type, and 4 were inconsistent of cardinality.
- **2 bugs were identified by the QUERY PARTITIONING oracle,** presenting the same pattern (content and size mismatch on aggregation methods). A distinctive element was the presence of the hint /*+AGG TO COP()*/, which tells TiDB to push the aggregation operations to the storage nodes (TiKV). The defects signal major problems for synchronization of the complex predicates between the computing level and the storage one.

4.4.4 Results for MariaDB and DuckDB

The MariaDB (introduced as an addition to the 2020 study) system was tested successfully using the NoREC and DQP oracles. It didn't registered any logic bugs, although the platform generated over 5.4 million queries, with a remarkable throughput of 16,671 q/s. This proves an excellent stability of the MariaDB optimizer on the logic routes tested by NoREC.

Similarly, DuckDB (which in the original work registered 72 bugs) registered 0 defect in the current campaign (on the TLP WHERE oracle). This result, obtained on a modern

version os the system validates the idea that the identified vulnerabilities by the fuzzing community in the previous years were solved, confirming the healthy cycle of software improvement.

4.5 Comparison

A differential fuzzing campaign is as efficient as the logic oracle that guides it. One of the secondary objectives of this study was comaprative evaluation of different oracles implemented in SQLancer (TLP, NoREC, PQS), in order to establish a correlation between the throughput and the efficiency in detecting logic defects.

The collected data through the orchestration platform showed multiple counterintuitive patterns, proving that a high volume of queries doesn't guarantee the vulnerabilities discovery.

4.5.1 TLP vs NoREC: quality vs quantity

The NoREC oracle is recognized for its high performance, because it needs a minimal overhead during the evaluation (just compares an optimized query with an regular one). The experimental data confirmed this advantage of performance: on SQLite, NoREC reached a throughput of 15,601 q/s (generating over 7,8 million queries), and MariaDB reached 16,671 q/s. With all that, on both systems, NoREC discovered 0 logic bugs.

On the other hand, the TLP WHERE oracle needed a more complex processing. Although MySQL's speed dropped to 5,600 q/s, TLP WHERE managed to identify 2 logic bugs. Even on SQLite, where its speed was comparable to NoREC (1,239 q/s), TLP WHERE exposed 1 logic bug which went completely unseen by the non-optimized testing methods. This comparison proves superiority to the ternary logic partitioning method in exploring deep execution threads of the optimizer, validating TLP as being a better analytic tool.

4.5.2 Power of specific oracles (HAVING and AGGREGATE)

A major conclusion of this study is the importance of developing some oracles dedicated to SQL specific clauses, areas that are mostly ignored by the general testing methods (like NoREC or PQS).

- **TLP AGGREGATE case (SQLite):** Aggregation functions are notoriously hard to test because of their nature which reduces the data dimensionality. Applying the TLP AGGREGATE oracle on SQLite recorded a very low throughput and a success rate of only 55 percent. With all that, it managed to identify 1 complex bug related to text conversion in the full-text indexation, an impossible scenario to reproduce with NoREC.
- **TLP HAVING case (TiDB):** The real potential of the oracles that are ultra-specialized was seen in testing the TiDB system. While the generic oracle TLP WHERE run with

3,554 q/s without any error, switching to TLP HAVING (which tests post-aggregation predicates) lead to the collapse of the success rate and the throughput, but showed a massive number of 13 bugs. Thus, although TLP HAVING was 3 times slower than TLP WHERE, it was infinitely more efficient in detecting errors on this distributed engine.

4.5.3 PQS ineffciency

The PQS oracle was evaluated on the systems that offered support for it in the SQLancer tool. Although it registered a respectable throughput (15,641 q/s on SQLite and 2,893 q/s on MySQL, with success rates of over 97 percent), PQS did not identified any bug in the testing campaigns. Poor performance of PQS compared to TLP suggests that, once with database maturing, the approach of data partitioning type is more capable of winning over the modern optimizers than the pivot based synthesis.

In conclusion, the comparative analysis outlines the necessity of the complementary usage of the oracles. While NoREC can be used for a fast smoke test, TLP oracles (especially the ones focused on complex clauses like HAVING or GROUP BY) represent the golden standard for discovering deep semantic vulnerabilities in the modern databases.

4.6 Threats to validity

Any empirical software engineering study has some internal limitations, dictated by the technology constraints, resources and the nature of the used instruments. In order to ensure an objective interpretation of the presented results in this chapter, it is needed a transparent analysis of the threats to the validity of the experiments, as well as the architectural limitations identified during the testing campaign.

4.6.1 Software rotting and tooling incompatibilities

The harshest limitation of the replication studies that implies third party tools is the software rot phenomenon. SQLancer, in spite of being a very powerful tool, is tightly coupled with the internal APIs and the database system configurations. The years brought major architectural changes in the databases, coming with fatal incompatibilities in SQLancer 2.0.0:

- **CockroachDB (v13.0.0):** Testing was completely compromised because the internal configuration parameter sql.metrics.statement.details.plan.collection.enabled, used by SQLancer for tooling, was deprecated and removed from the CockroachDB core in the newer versions.
- **PostgreSQL (v15.17):** The tool failed repeatedly, generating critical exceptions like NullPointerException right in the populating phase of the database. Although this invalidated the current results for PostgreSQL, the phenomenon proves the fragility of the maintenance of academic testing tools on the long run.

4.6.2 Time and computational resources constraints

Fuzzing is a stochastic process of state of space exploration. In the original 2020 study, SQLancer was run on dedicated servers during tens or hundreds of hours to exhaust the execution threads of the optimizers. In this study, because of resource limitations and in order to validate the agile orchestration capacity of the developed platform, the experiments were restrained to short time windows (5 to 10 minutes per oracle). Although this limitation reduced the absolute number of discovered vulnerabilities, their discovery in such a short time proves the differential testing efficiency. It is accepted as a threat to internal validity the fact that an extended run time could bring to light deeper bug classes.

4.6.3 Testing matrix asymmetry

The experimental design was affected by an uneven coverage of the oracles. SQLancer does not offer implementations for all the oracles on all the databases. For example:

- NoREC oracle was not available for MySQL, preventing a direct performance comparison with SQLite.
- The distributed TiDB system did not benefit of TLP GROUP BY or NoREC implementations. This asymmetry limits the possibility of a perfect crossed comparison between all the systems and all the oracles, the analysis being limited only to the compatible tuples.

4.6.4 Target system maturation

Another variable that influences the vulnerability quantity directly discovered is the version of the target systems. The study chose recent versions, published a few years after the original work. Open-source communities behind these engines have already integrated a big part of the reported bugs signaled by the researchers that used SQLancer in the past. The lack of bug discovery of new bugs on some systems does not represent a fail of the current methodology, but more like a proof of the maturing of the software systems and the efficiency of the testing tool.

Chapter 5

Conclusion and Future work

Ensuring the correctness of the database management systems remains one of the most critical and complex challenges in modern software engineering. Although differential fuzzing tools like SQLancer have proved their efficiency in detecting bugs, using them native, though the command line interfaces, present limitation in terms of traceability, data persistence and real time monitoring.

This work directly addressed this technological gap, proposing, designing and implementing from scratch a full-stack web platform for test orchestration. The decoupled architecture composed of a reactive front end interface, an asynchronous backend capable to manage operating system sub processes and a relational persistence layer has turned the isolated run of some scripts in an automated work flow, centralized and visually oriented. The platform enabled the easy launch of experiments, log parsing, as well as essential metrics (throughput, success rate, bug distribution) aggregation directly into an interactive dashboard.

Beyond the architectural and engineering contribution, the work included a practical testing campaign aimed to validate the proposed system. Although the experiments were constrained to short time windows, the orchestration platform showed a success, identifying 19 bugs. These defects varied from subtle omissions in the WHERE and AGGREGATE clauses on mature systems like SQLite and MySQL, all the way to a massive series of vulnerabilities on the distributed TiDB engine, triggered by the optimizers faulty interaction with the HAVING clause and the NATURAL RIGHT JOIN hints. The fact that the TLP HAVING oracle generated 13 errors on TiDB represent a valuable empirical contribution of this thesis, outlining the fact that new and complex systems remain prone to semantic errors in the post-aggregation logic.

In conclusion, the integration of the automated database testing into a dedicated observability framework, not only can make the researcher's work easier, but also streamlines the discovery and documentation of vulnerabilities in a structured and reproducible way.

Although the developed platform has reached its proposed objectives, the ecosystem of ensuring database quality is in a continuous evolution. The following directions are proposed for expanding and improving the current solution:

1. **Full dockerization:** Although in the current study's the target databases were isolated and efficiently instantiated using docker containers, the orchestration platform was running native. An immediate future direction proposes the react application and python server packaging into a unified docker-compose configuration. This decision could guarantee a higher level of portability, allowing anyone to run and replicate the studies with one command, canceling completely the dependencies and discrepancies at the operating system level.

2. **Multi-fuzzer integration:** The current platform was developed with SQLancer as it core. A natural development would be the abstraction of the integration layer from backend in order to support alternate fuzzing engines, like SQLsmit, Squirrel, or RAGS. Thus, the platform could turn from a simple SQLancer orchestrator into a centralized hub for testing database systems, allowing the user to compare not only the database engines, but also the generators efficiency.

3. **Active alerting and notifications:** Currently, bug identification is possible only through the dashboard. In order to sustain future fuzzing campaigns, that run over hundreds of hours, a webhooks module could be integrated. This system would notify the researchers automatically through communication platforms (like e-mail, slack, teams) exactly when the platform intercepts and validates a new bug in the terminal logs, optimizing considerably the reaction time.

Bibliography

[AAG+18] Yazeed Alabdulkarim, Marwan Almaymoni, Shahram Ghandeharizadeh, Haoyu Huang, and Hieu Nguyen. Testing database applications with polygraph. In *Proceedings of the 20th International Conference on Information Integration and Web-Based Applications & Services*, iiWAS2018, page 200–206, New York, NY, USA, 2018. Association for Computing Machinery.

[AKM18] Abdullah Alsharif, Gregory M. Kapfhammer, and Phil McMinn. Domino: Fast and effective test data generation for relational database schemas. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, pages 12–22, 2018.

[BD21] Ledia Bozo and Olta Dedej. In-memory database testing performance measurements in azure. volume 2872, page 61 – 66, 2021. Cited by: 0.

[CAD18] Aleksey Charapko, Ailidani Ailijiang, and Murat Demirbas. Adapting to access locality via live data migration in globally distributed datastores. In *2018 IEEE International Conference on Big Data (Big Data)*, pages 3321–3330, 2018.

[Cam18] Gabriel Campeanu. Gpu support for component-based development of embedded systems. 2018.

[CKL+18] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Comput. Surv.*, 51(1), January 2018.

[CSJ+14] Tse-Hsun Chen, Weiyi Shang, Zhen Ming Jiang, Ahmed E. Hassan, Mohamed Nasser, and Parminder Flora. Detecting performance anti-patterns for applications developed using object-relational mapping. Number CONFCODENUMBER, page 1001 – 1012, 2014. Cited by: 115.

[CSY+16] Tse-Hsun Chen, Weiyi Shang, Jinqiu Yang, Ahmed E. Hassan, Michael W. Godfrey, Mohamed Nasser, and Parminder Flora. An empirical study on the practice of maintaining object-relational mapping code in java systems. In *Proceedings of the 13th International Conference on Mining Software Repositories*, MSR '16, page 165–176, New York, NY, USA, 2016. Association for Computing Machinery.

[CZPC21] Davide Corradini, Amedeo Zampieri, Michele Pasqua, and Mariano Ceccato. Empirical comparison of black-box test case generation tools for restful apis. In *2021 IEEE 21st International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 226–236, 2021.

[DDP+23] Pouria Derakhshanfar, Xavier Devroey, Annibale Panichella, Andy Zaidman, and Arie van Deursen. Generating class-level integration tests using call site information. *IEEE Transactions on Software Engineering*, 49(4):2069–2087, 2023.

[GH19] Arief Ginanjar and Mokhamad Hendayun. Spring framework reliability investigation against database bridging layer using java platform. volume 161, page 1036 – 1045, 2019. Cited by: 9; All Open Access, Gold Open Access.

[Har93] Peter Harwood. The title of the work. Master's thesis, The school of the thesis, The address of the publisher, 7 1993. An optional note.

[Hon20] Yunguo Hong. Analysis on database testing technology of computer software development. In *2020 International Conference on Big Data Artificial Intelligence Software Engineering (ICBASE)*, pages 35–38, 2020.

[Jos93] Peter Joslin. *The title of the work*. PhD thesis, The school of the thesis, The address of the publisher, 7 1993. An optional note.

[KC07] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. 2, 01 2007.

[Kit04] Barbara Kitchenham. Procedures for performing systematic reviews. *Keele, UK, Keele Univ.*, 33, 08 2004.

[LPR22] Maurizio Leotta, Davide Paparella, and Filippo Ricca. Comparing the effectiveness of assertions with differential testing in the context of web testing. In Antonio Vallecillo, Joost Visser, and Ricardo Pérez-Castillo, editors, *Quality of Information and Communications Technology*, pages 108–124, Cham, 2022. Springer International Publishing.

[LWX+25] Shuang Liu, Ruifeng Wang, Yuanfeng Xie, Junjie Chen, Wei Lu, Xiao Zhang, Quanqing Xu, Chuanhui Yang, and Xiaoyong Du. A comprehensive study of bugs in relational dbms. *IEEE Transactions on Software Engineering*, pages 1–14, 2025.

[LZAO22] Xinyu Liu, Qi Zhou, Joy Arulraj, and Alessandro Orso. Automatic detection of performance bugs in database systems using equivalent queries. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, page 225–236, New York, NY, USA, 2022. Association for Computing Machinery.

[Min] Ministry for Primary Industries. Kina sea urchin regions in NZ. `http://fs.fish.govt.nz/Page.aspx?pk=7&sc=SUR`. Online; accessed 29 January 2014.

[MWK+16] Phil McMinn, Chris J. Wright, Cody Kinneer, Colton J. McCurdy, Michael Camara, and Gregory M. Kapfhammer. Schemaanalyst: Search-based test data generation for relational database schemas. In *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 586–590, 2016.

[Mö22] Jonas Möller. Differential testing: How to find differences between programs that mostly behave identically? GI SICHERHEIT 2022, 2022.

[RB12] Narayan Ramasubbu and Rajesh Krishna Balan. Overcoming the challenges in cost estimation for distributed software projects. In *Proceedings of the 34th International Conference on Software Engineering*, ICSE '12, page 91–101. IEEE Press, 2012.

[RB22] Paul Ralph and Sebastian Baltes. Paving the way for mature secondary research: the seven types of literature review. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2022, page 1632–1636, New York, NY, USA, 2022. Association for Computing Machinery.

[RG03] Raghu Ramakrishnan and Johannes Gehrke. *Database Management Systems*. McGraw-Hill, 3rd edition, 2003.

[RRT15] Romain Pierre Julien Robbes, D Röthlisberger, and É Tanter. Object-oriented software extensions in practice. *Empirical Software Engineering*, 20(3), 2015.

[RS20] Manuel Rigger and Zhendong Su. Finding bugs in database systems via query partitioning. *Proc. ACM Program. Lang.*, 4(OOPSLA), November 2020.

[SCA+21] Thodoris Sotiropoulos, Stefanos Chaliasos, Vaggelis Atlidakis, Dimitris Mitropoulos, and Diomidis Spinellis. Data-oriented differential testing of object-relational mapping systems. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1535–1547, 2021.

[Sha15] Sina Shamshiri. Automated unit test generation for evolving software. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2015, page 1038–1041, New York, NY, USA, 2015. Association for Computing Machinery.

[SJR+15] Sina Shamshiri, René Just, José Miguel Rojas, Gordon Fraser, Phil McMinn, and Andrea Arcuri. Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges (t). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 201–211, 2015.

[SKS20] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill Education, 7th edition, 2020.

[Som10] Ian Sommerville. *Software Engineering*. Addison-Wesley Publishing Company, USA, 9th edition, 2010.

[SV23] Yiding Sun and Romany Viju. Application of database testing techniques in web development. In Jemal H. Abawajy, Zheng Xu, Mohammed Atiquzzaman, and Xiaolu Zhang, editors, *Tenth International Conference on Applications and Techniques in Cyber Intelligence (ICATCI 2022)*, pages 311–318, Cham, 2023. Springer International Publishing.

[Tan24] Wei Tang. Development of computer application system and database testing based on data encryption technology. *Applied Mathematics and Nonlinear Sciences*, 9(1), 2024. Cited by: 1; All Open Access, Gold Open Access.

[TSM+20] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, Paul Bardea, Amruta Ranade, Ben Darnell, Bram Gruneir, Justin Jaffray, Lucy Zhang, and Peter Mattis. Cockroachdb: The resilient geo-distributed sql database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, SIGMOD '20, page 1493–1509, New York, NY, USA, 2020. Association for Computing Machinery.

[Wan20] MengXia Wang. Analysis of database programming technology in computer software engineering. In *2020 International Conference on Computer Communication and Network Security (CCNS)*, pages 1–4, 2020.

[WLM+21] Chaolun Wang, Yuan Liu, Pengwei Ma, Jiafeng Tian, Siyuan Liu, Minjing Zhong, and Chunyu Jiang. Research and implementation of an analytical database testing platform in telecommunication industry. In *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 1328–1333, 2021.

[WLZ+23] Qingshuai Wang, Yuming Li, Rong Zhang, Ke Shu, Zhenjie Zhang, and Aoying Zhou. A scalable query-aware enormous database generator for database evaluation. *IEEE Transactions on Knowledge and Data Engineering*, 35(5):4395–4410, 2023.

[YYS+18] Junwen Yang, Cong Yan, Pranav Subramaniam, Shan Lu, and Alvin Cheung. How not to structure your database-backed web applications: A study of performance bugs in the wild. In *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*, pages 800–810, 2018.

[ZCH+20] Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. Squirrel: Testing database management systems with language validity and coverage feedback. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, CCS '20, page 955–970, New York, NY, USA, 2020. Association for Computing Machinery.

[ZSS+18] Irene Zhang, Naveen Kr. Sharma, Adriana Szekeres, Arvind Krishnamurthy, and Dan R. K. Ports. Building consistent transactions with inconsistent replication. *ACM Trans. Comput. Syst.*, 35(4), December 2018.